

Neural Networks

Introduction to Artificial Intelligence

Emily Sheetz

Assumptions on Content

Throughout this lecture, we assume you have some familiarity with:

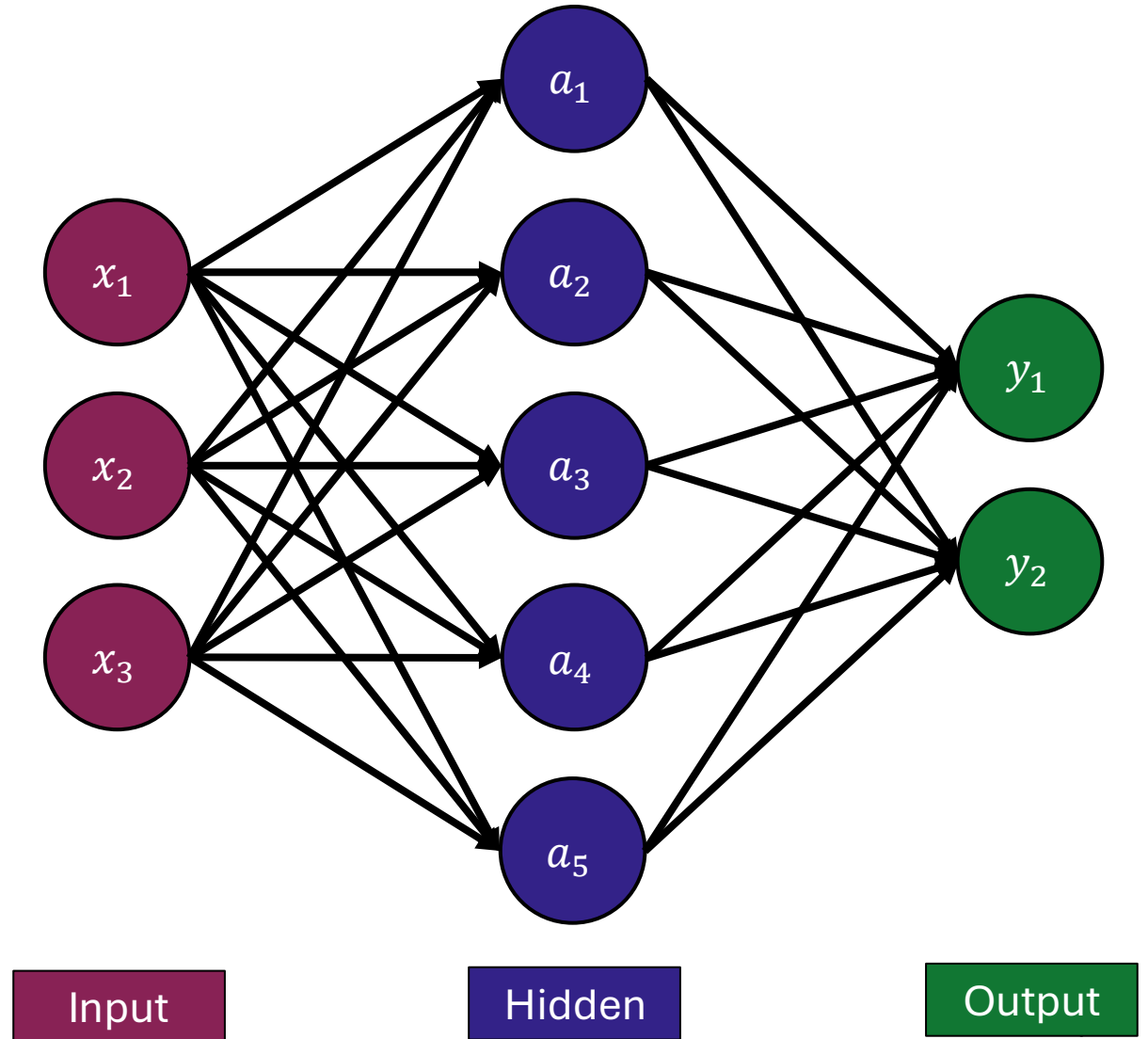
- **Linear Algebra** (dot product, transpose, matrix multiplication)
 - [Linear Algebra Review for Machine Learning](#), Sections 1, 2, and 4
- **Calculus III** (partial derivatives)
 - [Intro to Partial Derivatives and Gradient Vectors](#) and/or [Derivatives for Machine Learning](#)

Please refer to the linked resources if you need a review!

Practice with Neural Networks

Neural Networks

- Learn a function to map *inputs or features* to *outputs or predictions* using any number of *hidden layers*

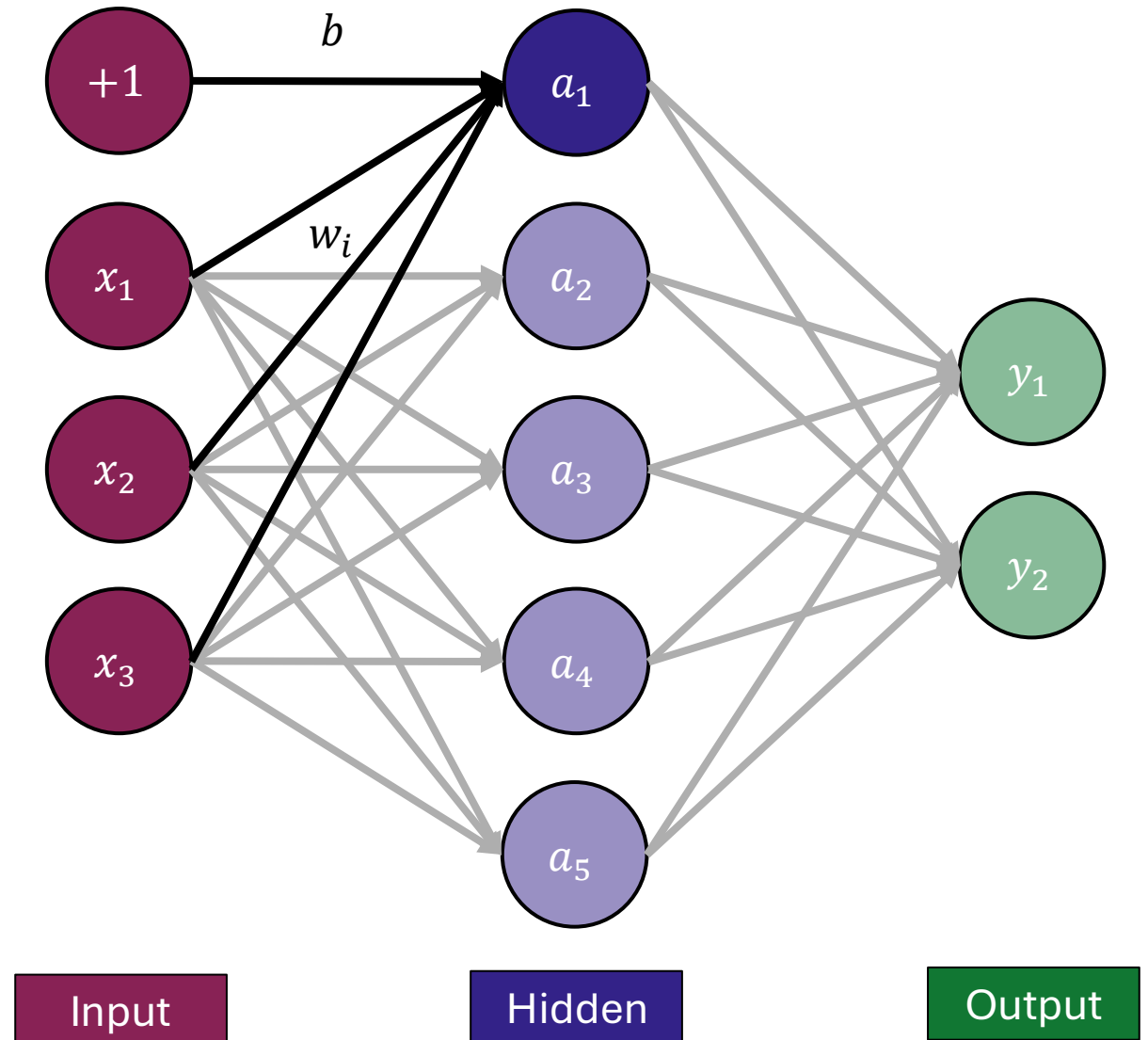


Neural Networks

- Learn a function to map *inputs or features* to *outputs or predictions* using any number of *hidden layers*
- Each *node* or *neuron* performs a weighted sum on its inputs, and activates the result

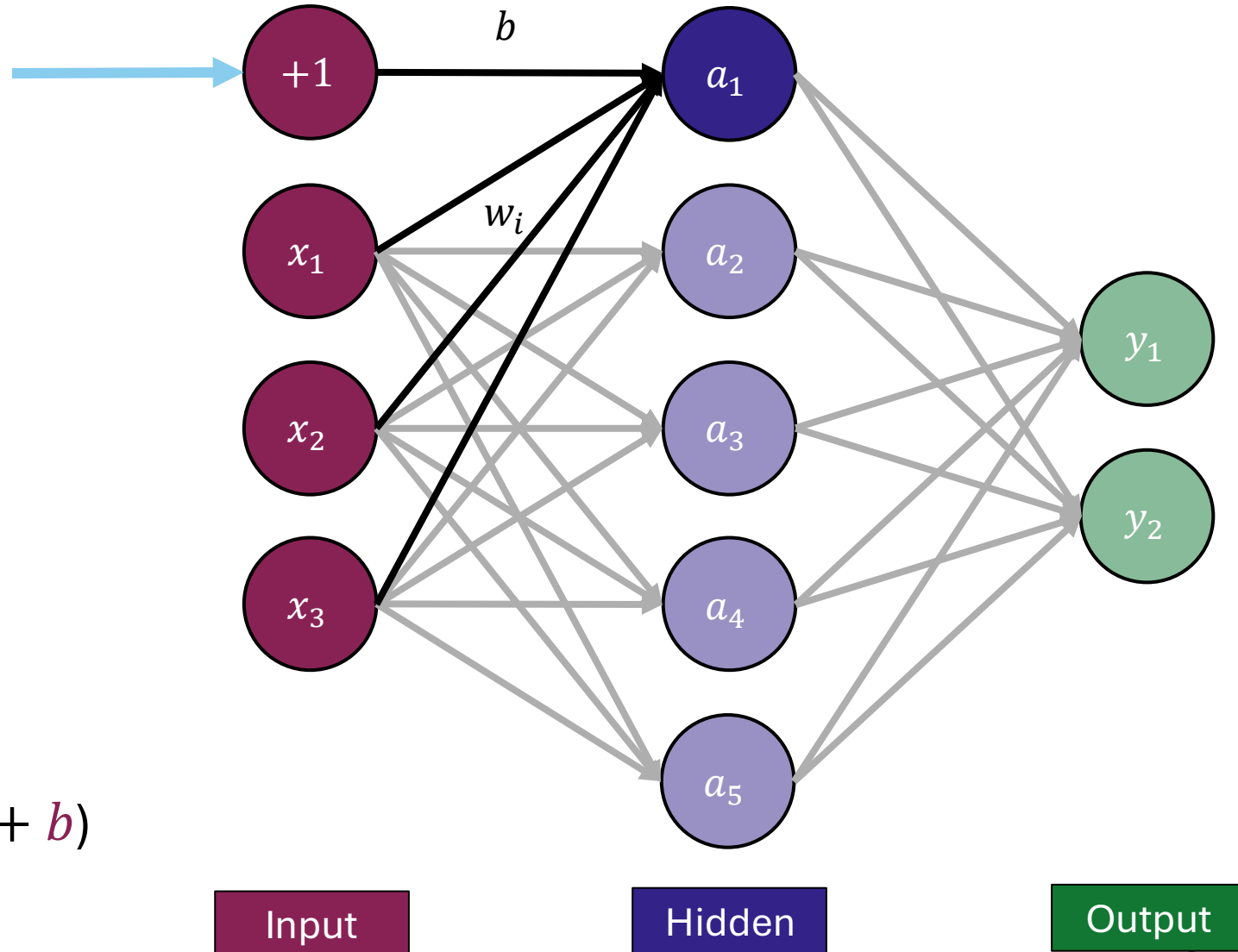
$$a_1 = g(\mathbf{w} \cdot \mathbf{x} + \mathbf{b})$$

$$a_1 = g(w_1x_1 + w_2x_2 + w_3x_3 + b)$$



Neural Networks: Terminology Review

What is this input called?



$$a_1 = g(\mathbf{w} \cdot \mathbf{x} + \mathbf{b})$$

$$a_1 = g(w_1x_1 + w_2x_2 + w_3x_3 + \mathbf{b})$$

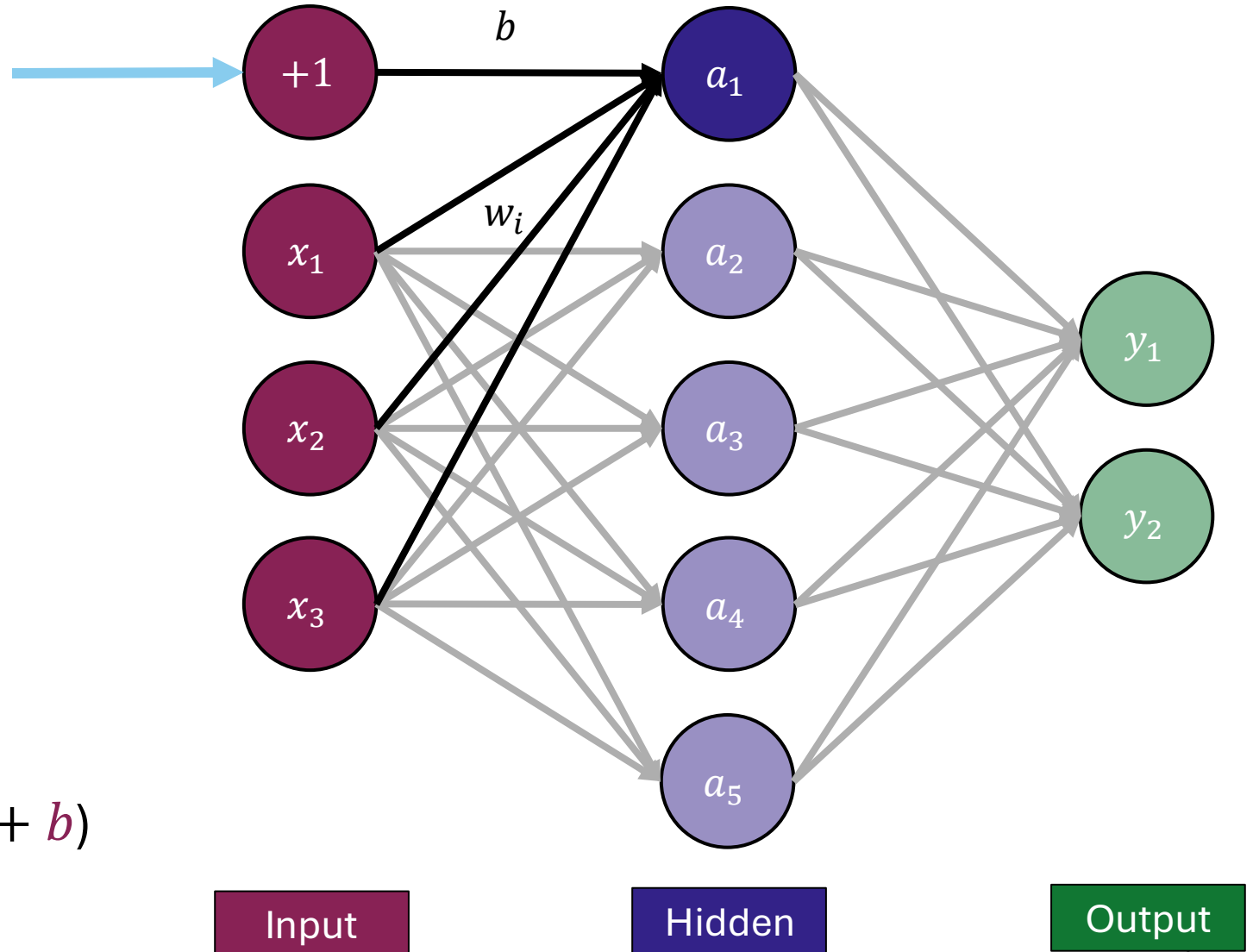
Neural Networks: Terminology Review

What is this input called?

The **bias input**

$$a_1 = g(\mathbf{w} \cdot \mathbf{x} + \mathbf{b})$$

$$a_1 = g(w_1x_1 + w_2x_2 + w_3x_3 + \mathbf{b})$$

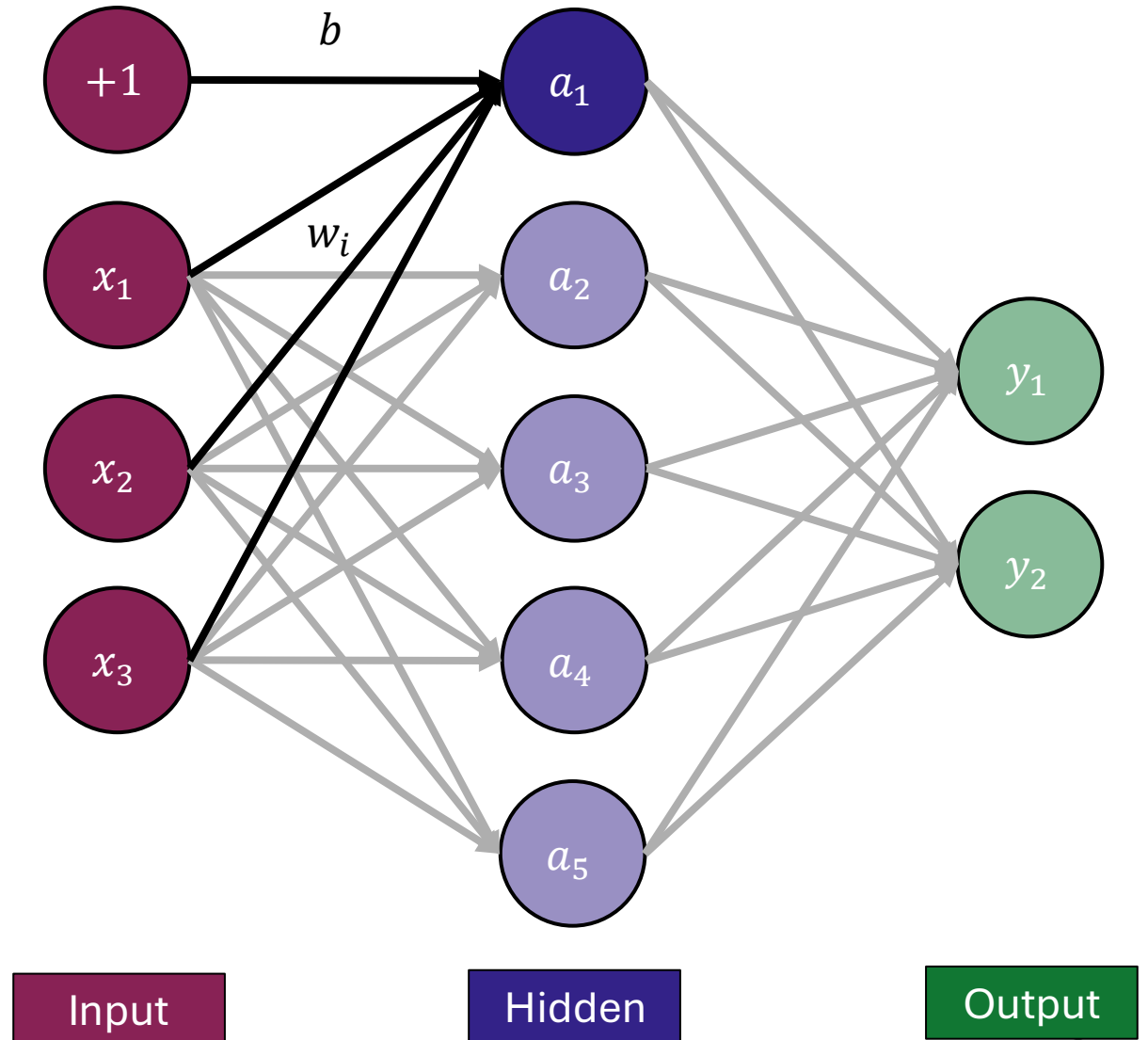


Neural Networks: Terminology Review

What is this function called?

$$a_1 = g(\mathbf{w} \cdot \mathbf{x} + \mathbf{b})$$

$$a_1 = g(w_1x_1 + w_2x_2 + w_3x_3 + b)$$

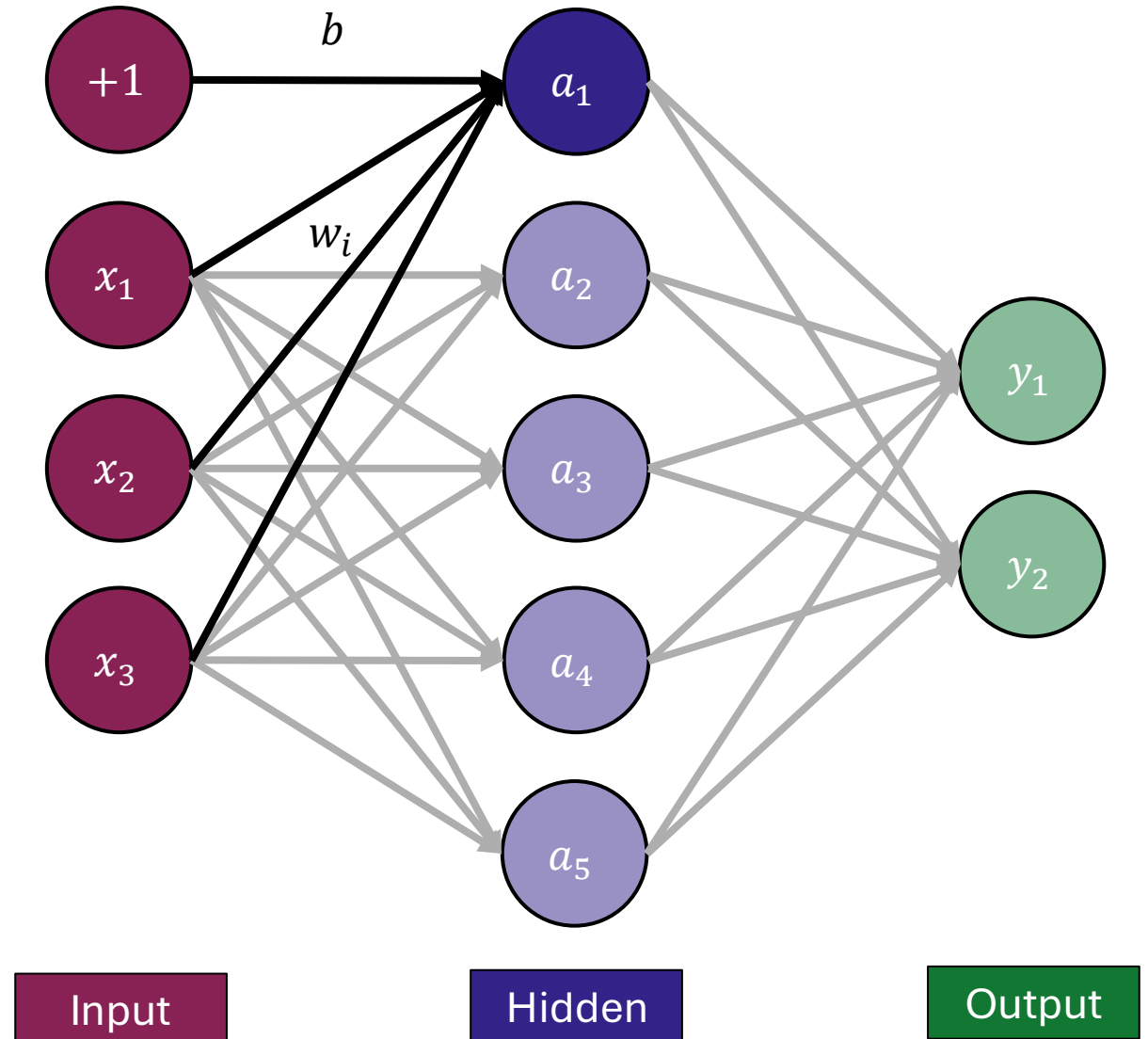


Neural Networks: Terminology Review

What is this function called?

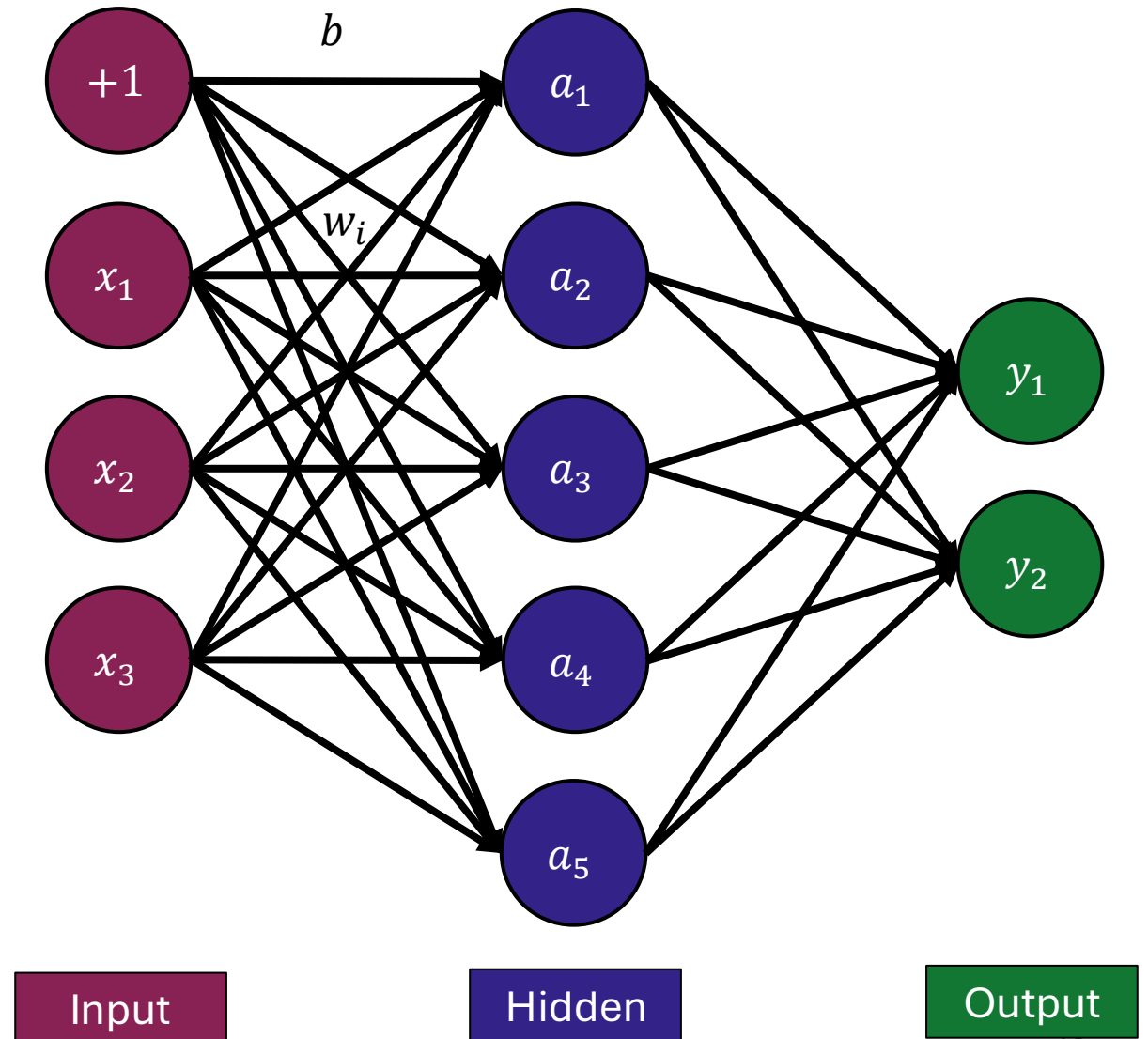
The **activation function**, which provides additional control to the node output. There are many possible activation functions.

$a_1 = g(\mathbf{w} \cdot \mathbf{x} + \mathbf{b})$
 $a_1 = g(w_1x_1 + w_2x_2 + w_3x_3 + b)$



Neural Networks

- Learn a function to map *inputs or features* to *outputs or predictions* using any number of *hidden layers*
- The network learns the *weights* to minimize the *error* between the predicted $\hat{y}$ and actual y outputs



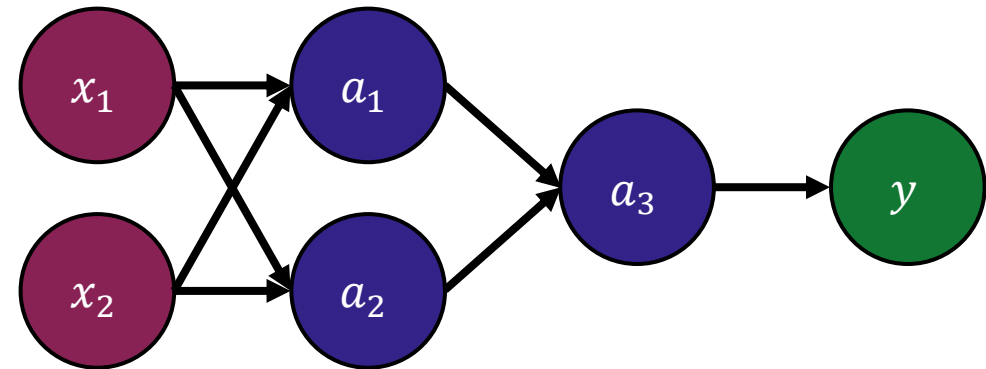
Example Problem

Suppose we want to open a restaurant. We are going to train a multi-layer neural network to predict if a new menu item will do well. We collected data on different foods, features, and whether the food item is enjoyed, rated on a $[0,1]$ scale.

We are going to use the architecture on the right. Since our outputs are bounded $[0,1]$, we will use the sigmoid activation.

$$g(z) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Food	x_1	x_2	Rating
Mousse	1	1	0.1
Pie	3	2	0.5
Ice Cream	2	6	0.2
Sorbet	3	4	0.4
Cheesecake	5	3	0.9



$W^{[l]}, b^{[l]}$: weights and bias on layer l
 $z^{[l]}$: weighted sum of inputs on layer l ,
 $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$
 $g^{[l]}$: activation function on layer l
 $a^{[l]}$: output of activation function on layer l ,
 $a^{[l]} = g(z^{[l]})$

General Form of NN Output

What does the output from this neural network (NN) generally look like?

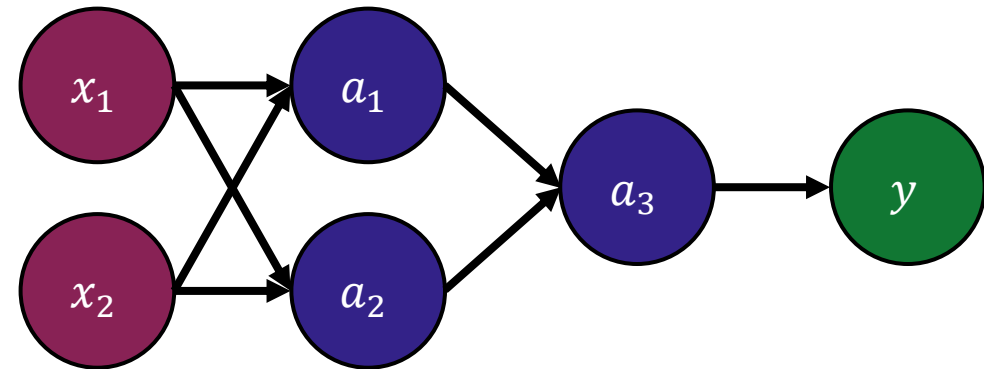
We can work from the output layer:

$$\hat{y} = a^{[2]} = g^{[2]}(z^{[2]})$$

$$\hat{y} = \sigma(W^{[2]}a^{[1]} + b^{[2]}) = \sigma(W^{[2]}g^{[1]}(z^{[1]}) + b^{[2]})$$

$$\hat{y} = \sigma(W^{[2]}\sigma(W^{[1]}a^{[0]} + b^{[1]}) + b^{[2]})$$

Food	x_1	x_2	Rating
Mousse	1	1	0.1
Pie	3	2	0.5
Ice Cream	2	6	0.2
Sorbet	3	4	0.4
Cheesecake	5	3	0.9



$W^{[l]}, b^{[l]}$: weights and bias on layer l

$z^{[l]}$: weighted sum of inputs on layer l ,

$$z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$$

$g^{[l]}$: activation function on layer l

$a^{[l]}$: output of activation function on layer l ,

$$a^{[l]} = g(z^{[l]})$$

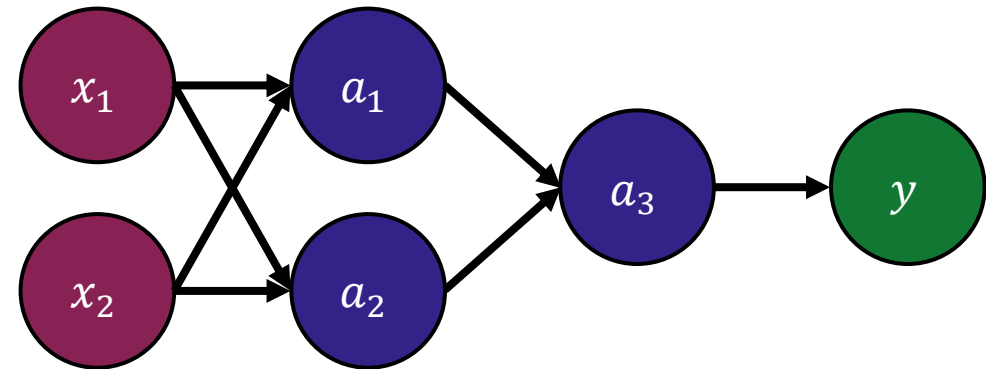
Forward Pass

What is the output when the data point for **Ice Cream** passes through the NN?

Use initial weights:

$$W^{[1]} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}, b^{[1]} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$
$$W^{[2]} = \begin{bmatrix} 1 & 1 \end{bmatrix}, b^{[2]} = \begin{bmatrix} 1 \end{bmatrix}$$

Food	x_1	x_2	Rating
Mousse	1	1	0.1
Pie	3	2	0.5
Ice Cream	2	6	0.2
Sorbet	3	4	0.4
Cheesecake	5	3	0.9



$W^{[l]}, b^{[l]}$: weights and bias on layer l
 $z^{[l]}$: weighted sum of inputs on layer l ,
 $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$
 $g^{[l]}$: activation function on layer l
 $a^{[l]}$: output of activation function on layer l ,
 $a^{[l]} = g(z^{[l]})$

Forward Pass

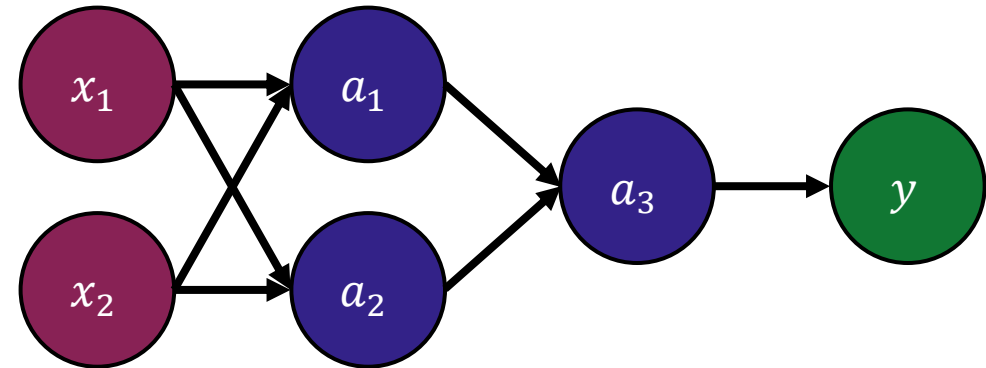
What is the output when the data point for **Ice Cream** passes through the NN?

$$W^{[1]} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}, b^{[1]} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$
$$W^{[2]} = \begin{bmatrix} 1 & 1 \end{bmatrix}, b^{[2]} = \begin{bmatrix} 1 \end{bmatrix}$$

Using general form:

$$\hat{y} = \sigma(W^{[2]} \sigma(W^{[1]} a^{[0]} + b^{[1]}) + b^{[2]})$$

Food	x_1	x_2	Rating
Mousse	1	1	0.1
Pie	3	2	0.5
Ice Cream	2	6	0.2
Sorbet	3	4	0.4
Cheesecake	5	3	0.9



$W^{[l]}, b^{[l]}$: weights and bias on layer l
 $z^{[l]}$: weighted sum of inputs on layer l ,
 $z^{[l]} = W^{[l]} a^{[l-1]} + b^{[l]}$
 $g^{[l]}$: activation function on layer l
 $a^{[l]}$: output of activation function on layer l ,
 $a^{[l]} = g(z^{[l]})$

Forward Pass

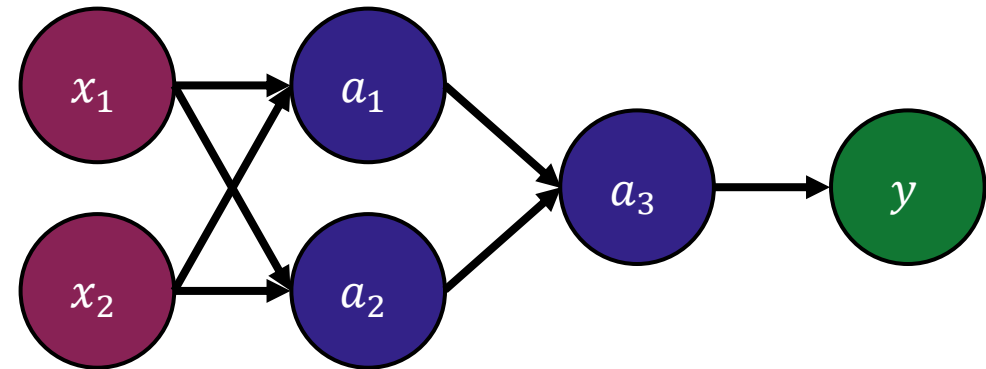
What is the output when the data point for **Ice Cream** passes through the NN?

$$W^{[1]} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}, b^{[1]} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$
$$W^{[2]} = \begin{bmatrix} 1 & 1 \end{bmatrix}, b^{[2]} = \begin{bmatrix} 1 \end{bmatrix}$$

Substitute data:

$$\hat{y} = \sigma(W^{[2]} \sigma(W^{[1]} \begin{bmatrix} 2 \\ 6 \end{bmatrix} + b^{[1]}) + b^{[2]})$$

Food	x_1	x_2	Rating
Mousse	1	1	0.1
Pie	3	2	0.5
Ice Cream	2	6	0.2
Sorbet	3	4	0.4
Cheesecake	5	3	0.9



$W^{[l]}, b^{[l]}$: weights and bias on layer l
 $z^{[l]}$: weighted sum of inputs on layer l ,
 $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$
 $g^{[l]}$: activation function on layer l
 $a^{[l]}$: output of activation function on layer l ,
 $a^{[l]} = g(z^{[l]})$

Forward Pass

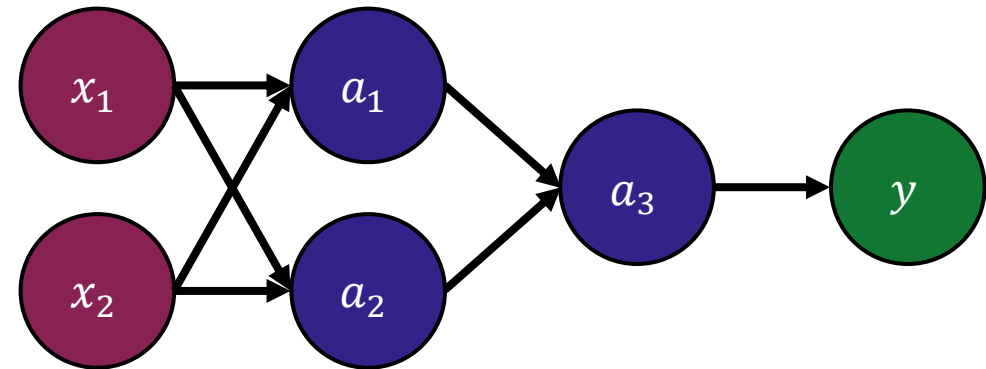
What is the output when the data point for **Ice Cream** passes through the NN?

$$W^{[1]} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}, b^{[1]} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$
$$W^{[2]} = \begin{bmatrix} 1 & 1 \end{bmatrix}, b^{[2]} = \begin{bmatrix} 1 \end{bmatrix}$$

Substitute weights and biases:

$$\hat{y} = \sigma([1 \quad 1] \sigma(\begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} 2 \\ 6 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \end{bmatrix}) + [1])$$

Food	x_1	x_2	Rating
Mousse	1	1	0.1
Pie	3	2	0.5
Ice Cream	2	6	0.2
Sorbet	3	4	0.4
Cheesecake	5	3	0.9



$W^{[l]}, b^{[l]}$: weights and bias on layer l

$z^{[l]}$: weighted sum of inputs on layer l ,

$$z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$$

$g^{[l]}$: activation function on layer l

$a^{[l]}$: output of activation function on layer l ,

$$a^{[l]} = g(z^{[l]})$$

Forward Pass

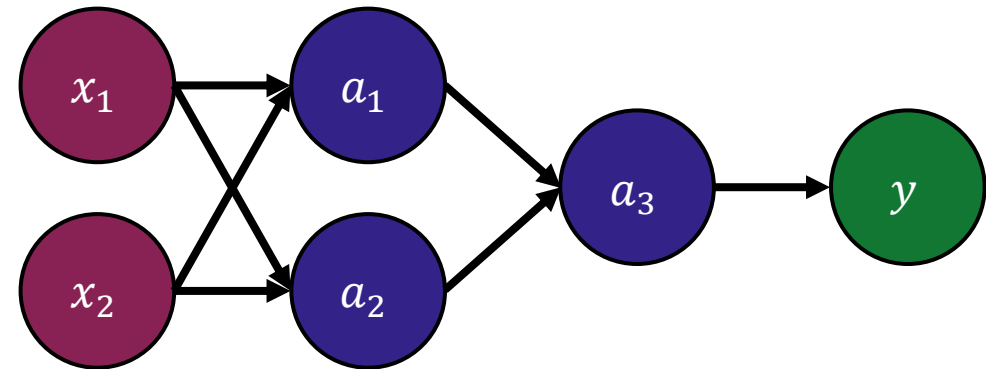
What is the output when the data point for **Ice Cream** passes through the NN?

$$W^{[1]} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}, b^{[1]} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$
$$W^{[2]} = \begin{bmatrix} 1 & 1 \end{bmatrix}, b^{[2]} = \begin{bmatrix} 1 \end{bmatrix}$$

Simplify first layer (weights):

$$\hat{y} = \sigma([1 \quad 1] \sigma\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \end{bmatrix}\right) + [1])$$

Food	x_1	x_2	Rating
Mousse	1	1	0.1
Pie	3	2	0.5
Ice Cream	2	6	0.2
Sorbet	3	4	0.4
Cheesecake	5	3	0.9



$W^{[l]}, b^{[l]}$: weights and bias on layer l

$z^{[l]}$: weighted sum of inputs on layer l ,

$$z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$$

$g^{[l]}$: activation function on layer l

$a^{[l]}$: output of activation function on layer l ,

$$a^{[l]} = g(z^{[l]})$$

Forward Pass

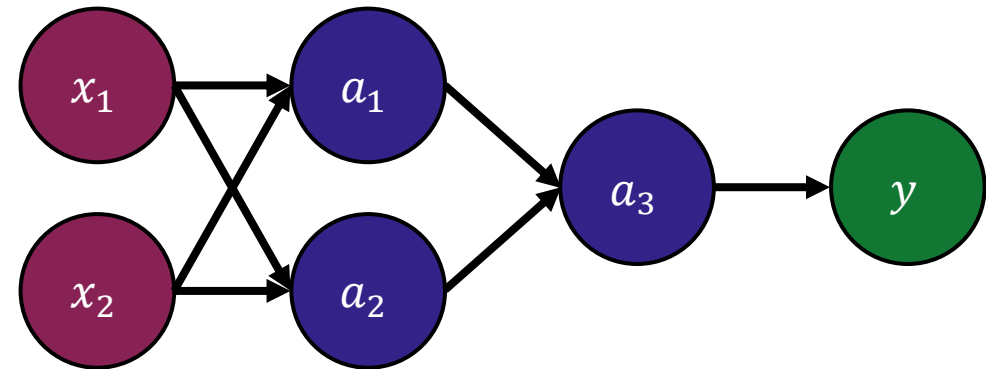
What is the output when the data point for **Ice Cream** passes through the NN?

$$W^{[1]} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}, b^{[1]} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$
$$W^{[2]} = \begin{bmatrix} 1 & 1 \end{bmatrix}, b^{[2]} = \begin{bmatrix} 1 \end{bmatrix}$$

Simplify first layer (bias):

$$\hat{y} = \sigma([1 \quad 1]\sigma\left(\begin{bmatrix} 1 \\ 1 \end{bmatrix}\right) + [1])$$

Food	x_1	x_2	Rating
Mousse	1	1	0.1
Pie	3	2	0.5
Ice Cream	2	6	0.2
Sorbet	3	4	0.4
Cheesecake	5	3	0.9



$W^{[l]}, b^{[l]}$: weights and bias on layer l

$z^{[l]}$: weighted sum of inputs on layer l ,

$$z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$$

$g^{[l]}$: activation function on layer l

$a^{[l]}$: output of activation function on layer l ,

$$a^{[l]} = g(z^{[l]})$$

Forward Pass

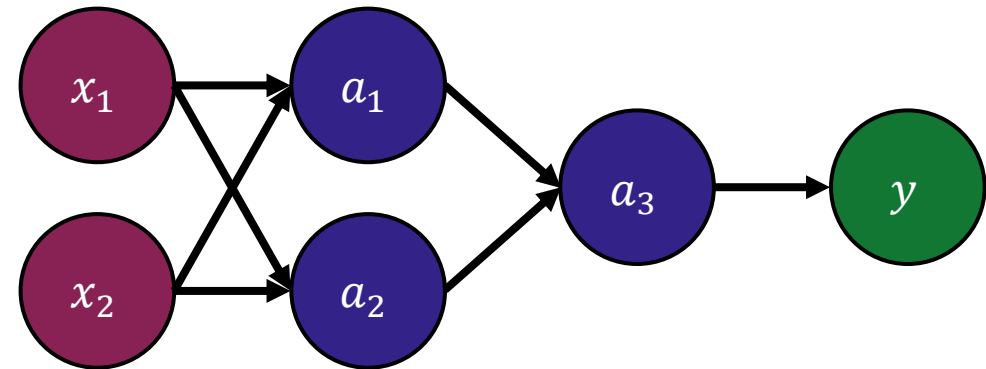
What is the output when the data point for **Ice Cream** passes through the NN?

$$W^{[1]} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}, b^{[1]} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$
$$W^{[2]} = \begin{bmatrix} 1 & 1 \end{bmatrix}, b^{[2]} = \begin{bmatrix} 1 \end{bmatrix}$$

Simplify first layer (activation):

$$\hat{y} = \sigma\left(\begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} \frac{1}{1 + e^{-1}} \\ 1 \\ \frac{1}{1 + e^{-1}} \end{bmatrix} + \begin{bmatrix} 1 \end{bmatrix}\right)$$

Food	x_1	x_2	Rating
Mousse	1	1	0.1
Pie	3	2	0.5
Ice Cream	2	6	0.2
Sorbet	3	4	0.4
Cheesecake	5	3	0.9



$W^{[l]}, b^{[l]}$: weights and bias on layer l
 $z^{[l]}$: weighted sum of inputs on layer l ,
 $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$
 $g^{[l]}$: activation function on layer l
 $a^{[l]}$: output of activation function on layer l ,
 $a^{[l]} = g(z^{[l]})$

Forward Pass

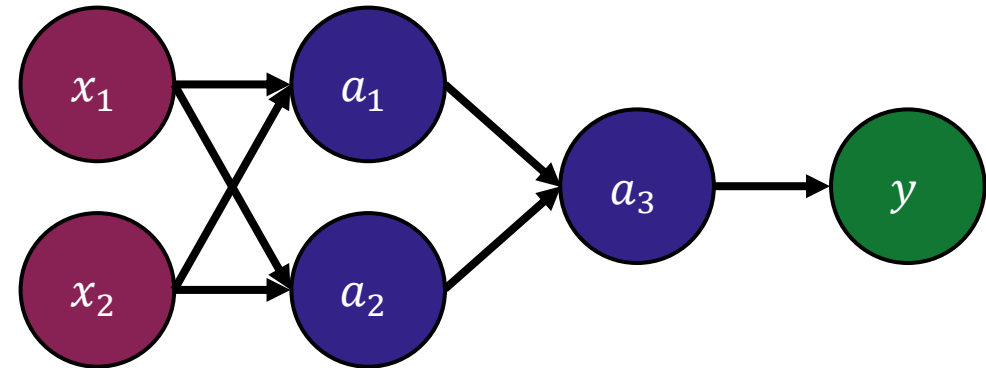
What is the output when the data point for **Ice Cream** passes through the NN?

$$W^{[1]} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}, b^{[1]} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$
$$W^{[2]} = \begin{bmatrix} 1 & 1 \end{bmatrix}, b^{[2]} = \begin{bmatrix} 1 \end{bmatrix}$$

Simplify first layer (activation):

$$\hat{y} = \sigma\left(\begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} 0.73106 \\ 0.73106 \end{bmatrix} + \begin{bmatrix} 1 \end{bmatrix}\right)$$

Food	x_1	x_2	Rating
Mousse	1	1	0.1
Pie	3	2	0.5
Ice Cream	2	6	0.2
Sorbet	3	4	0.4
Cheesecake	5	3	0.9



$W^{[l]}, b^{[l]}$: weights and bias on layer l

$z^{[l]}$: weighted sum of inputs on layer l ,

$$z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$$

$g^{[l]}$: activation function on layer l

$a^{[l]}$: output of activation function on layer l ,

$$a^{[l]} = g(z^{[l]})$$

Forward Pass

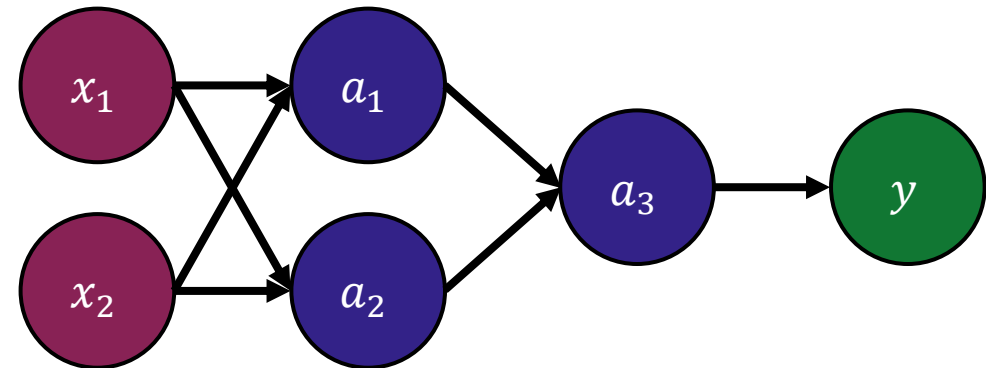
What is the output when the data point for **Ice Cream** passes through the NN?

$$W^{[1]} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}, b^{[1]} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$
$$W^{[2]} = \begin{bmatrix} 1 & 1 \end{bmatrix}, b^{[2]} = \begin{bmatrix} 1 \end{bmatrix}$$

Simplify second layer (weights):

$$\hat{y} = \sigma([1.46212] + [1])$$

Food	x_1	x_2	Rating
Mousse	1	1	0.1
Pie	3	2	0.5
Ice Cream	2	6	0.2
Sorbet	3	4	0.4
Cheesecake	5	3	0.9



$W^{[l]}, b^{[l]}$: weights and bias on layer l

$z^{[l]}$: weighted sum of inputs on layer l ,

$$z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$$

$g^{[l]}$: activation function on layer l

$a^{[l]}$: output of activation function on layer l ,

$$a^{[l]} = g(z^{[l]})$$

Forward Pass

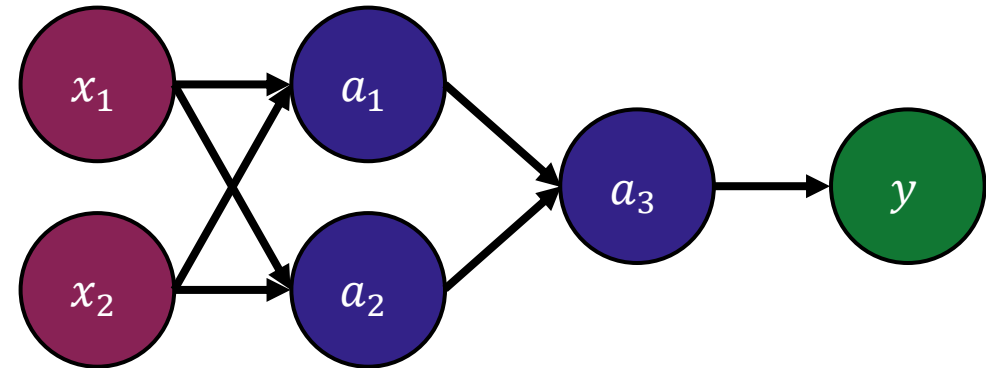
What is the output when the data point for **Ice Cream** passes through the NN?

$$W^{[1]} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}, b^{[1]} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$
$$W^{[2]} = \begin{bmatrix} 1 & 1 \end{bmatrix}, b^{[2]} = \begin{bmatrix} 1 \end{bmatrix}$$

Simplify second layer (bias):

$$\hat{y} = \sigma([2.46212])$$

Food	x_1	x_2	Rating
Mousse	1	1	0.1
Pie	3	2	0.5
Ice Cream	2	6	0.2
Sorbet	3	4	0.4
Cheesecake	5	3	0.9



$W^{[l]}, b^{[l]}$: weights and bias on layer l
 $z^{[l]}$: weighted sum of inputs on layer l ,
 $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$
 $g^{[l]}$: activation function on layer l
 $a^{[l]}$: output of activation function on layer l ,
 $a^{[l]} = g(z^{[l]})$

Forward Pass

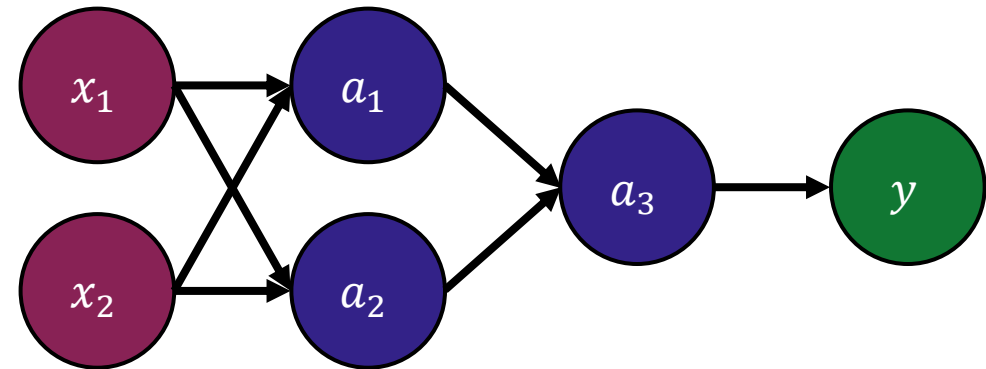
What is the output when the data point for **Ice Cream** passes through the NN?

$$W^{[1]} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}, b^{[1]} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$
$$W^{[2]} = \begin{bmatrix} 1 & 1 \end{bmatrix}, b^{[2]} = \begin{bmatrix} 1 \end{bmatrix}$$

Simplify second layer (activation):

$$\hat{y} = \left[\frac{1}{1 + e^{-2.46212}} \right]$$

Food	x_1	x_2	Rating
Mousse	1	1	0.1
Pie	3	2	0.5
Ice Cream	2	6	0.2
Sorbet	3	4	0.4
Cheesecake	5	3	0.9



$W^{[l]}, b^{[l]}$: weights and bias on layer l
 $z^{[l]}$: weighted sum of inputs on layer l ,
 $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$
 $g^{[l]}$: activation function on layer l
 $a^{[l]}$: output of activation function on layer l ,
 $a^{[l]} = g(z^{[l]})$

Forward Pass

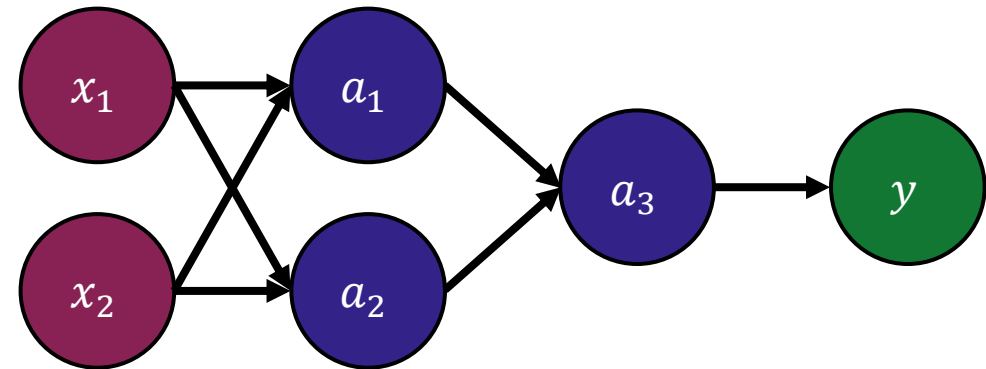
What is the output when the data point for **Ice Cream** passes through the NN?

$$W^{[1]} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}, b^{[1]} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$
$$W^{[2]} = \begin{bmatrix} 1 & 1 \end{bmatrix}, b^{[2]} = \begin{bmatrix} 1 \end{bmatrix}$$

Network prediction:

$$\hat{y} = [0.92144]$$

Food	x_1	x_2	Rating
Mousse	1	1	0.1
Pie	3	2	0.5
Ice Cream	2	6	0.2
Sorbet	3	4	0.4
Cheesecake	5	3	0.9



$W^{[l]}, b^{[l]}$: weights and bias on layer l

$z^{[l]}$: weighted sum of inputs on layer l ,

$$z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$$

$g^{[l]}$: activation function on layer l

$a^{[l]}$: output of activation function on layer l ,

$$a^{[l]} = g(z^{[l]})$$

Forward Pass: Discussion

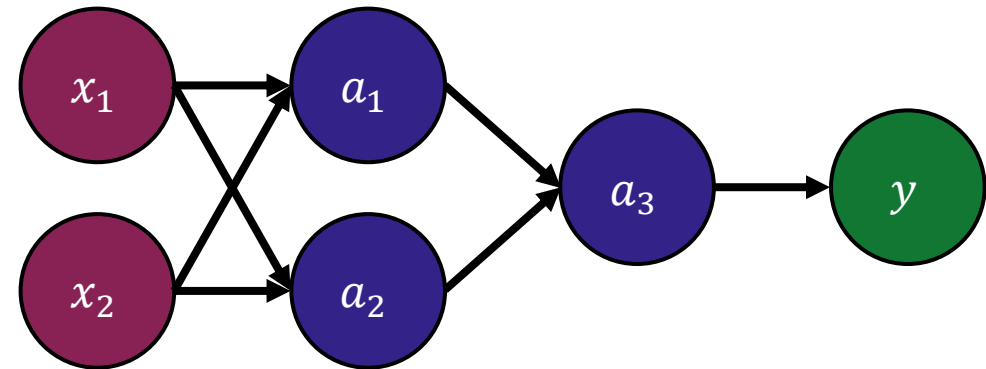
What is the output when the data point for **Ice Cream** passes through the NN?

$$\hat{y} = [0.92144]$$

Coming back to our goal: What does this value tell us? How do we interpret this value in the context of our problem?

This is the **predicted rating** for **Ice Cream** from customers in our new restaurant!

Food	x_1	x_2	Rating
Mousse	1	1	0.1
Pie	3	2	0.5
Ice Cream	2	6	0.2
Sorbet	3	4	0.4
Cheesecake	5	3	0.9

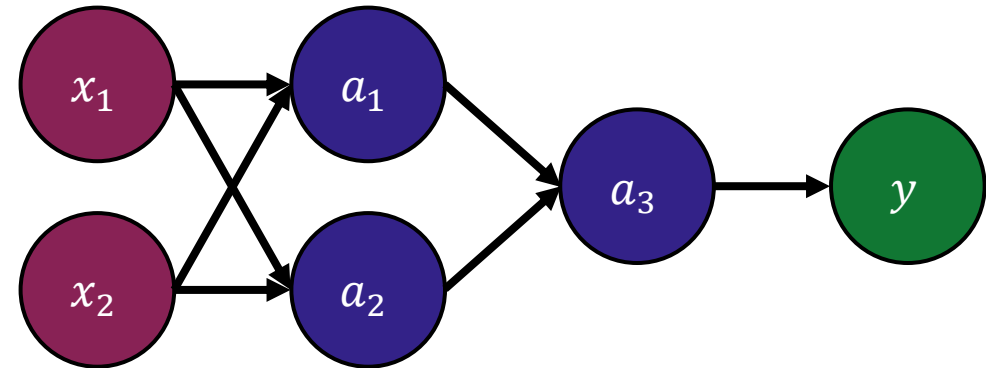


$W^{[l]}, b^{[l]}$: weights and bias on layer l
 $z^{[l]}$: weighted sum of inputs on layer l ,
 $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$
 $g^{[l]}$: activation function on layer l
 $a^{[l]}$: output of activation function on layer l ,
 $a^{[l]} = g(z^{[l]})$

Backpropagation

Based on our result from the **Ice Cream** data point, update the NN weights using **stochastic gradient descent**.

Food	x_1	x_2	Rating
Mousse	1	1	0.1
Pie	3	2	0.5
Ice Cream	2	6	0.2
Sorbet	3	4	0.4
Cheesecake	5	3	0.9



$W^{[l]}, b^{[l]}$: weights and bias on layer l

$z^{[l]}$: weighted sum of inputs on layer l ,

$$z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$$

$g^{[l]}$: activation function on layer l

$a^{[l]}$: output of activation function on layer l ,

$$a^{[l]} = g(z^{[l]})$$

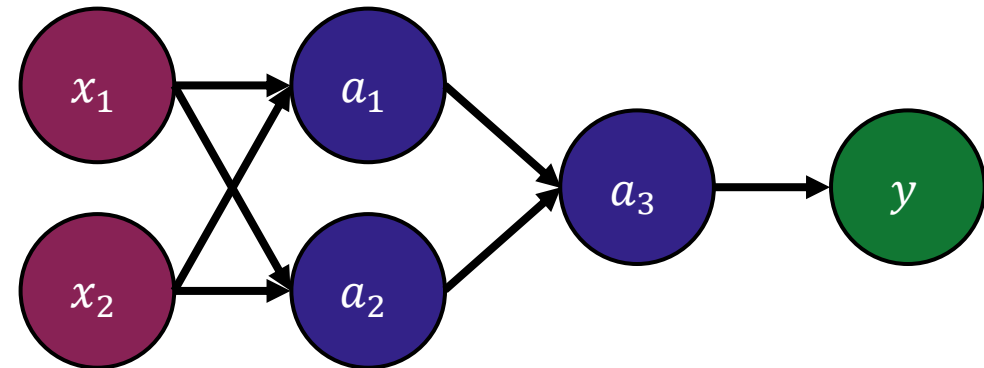
Backpropagation: Key Term

Based on our result from the **Ice Cream** data point, update the NN weights using **stochastic gradient descent**.

What do we mean by **stochastic** gradient descent?

Stochastic gradient descent (SGD) updates the weights based on **one** data point. Here, we want to update the weights just based on **Ice Cream**.

Food	x_1	x_2	Rating
Mousse	1	1	0.1
Pie	3	2	0.5
Ice Cream	2	6	0.2
Sorbet	3	4	0.4
Cheesecake	5	3	0.9



$W^{[l]}, b^{[l]}$: weights and bias on layer l
 $z^{[l]}$: weighted sum of inputs on layer l ,
$$z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$$
 $g^{[l]}$: activation function on layer l
 $a^{[l]}$: output of activation function on layer l ,
$$a^{[l]} = g(z^{[l]})$$

Backpropagation

Based on our result from the **Ice Cream** data point, update the NN weights using **stochastic gradient descent**.

Recall our weight update rule:

New weight

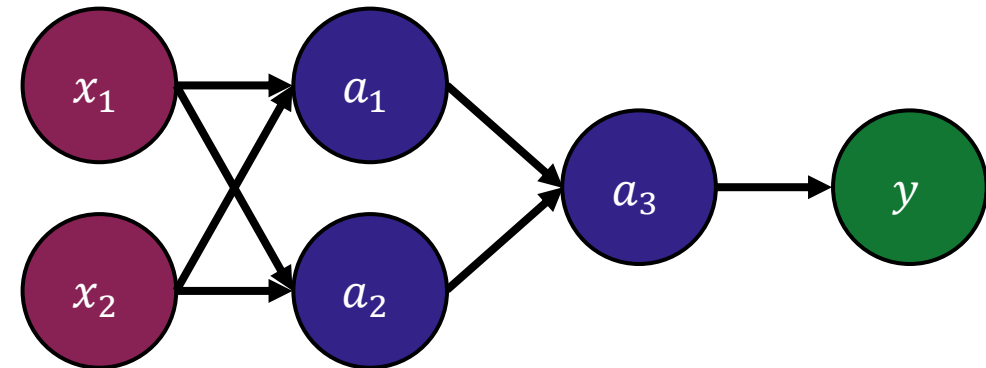
Learning rate

$$w_i \leftarrow w_i - \alpha \frac{\partial}{\partial w_i} \text{Loss}(\mathbf{w})$$

Current weight

Partial derivative of loss with respect to weight w_i evaluated with current network parameters $\mathbf{w}$

Food	x_1	x_2	Rating
Mousse	1	1	0.1
Pie	3	2	0.5
Ice Cream	2	6	0.2
Sorbet	3	4	0.4
Cheesecake	5	3	0.9



$W^{[l]}, b^{[l]}$: weights and bias on layer l
 $z^{[l]}$: weighted sum of inputs on layer l ,
 $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$
 $g^{[l]}$: activation function on layer l
 $a^{[l]}$: output of activation function on layer l ,
 $a^{[l]} = g(z^{[l]})$

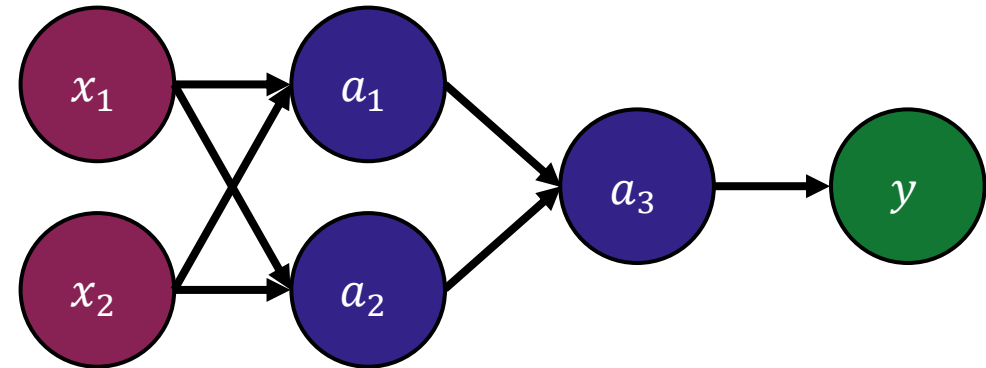
Backpropagation

Based on our result from the **Ice Cream** data point, update the NN weights using **stochastic gradient descent**.

We'll compute the partial derivative of the loss function: $L_2(y, \hat{y}) = (y - \hat{y})^2$

$$w_i \leftarrow w_i - \alpha \frac{\partial}{\partial w_i} L_2(\mathbf{w})$$

Food	x_1	x_2	Rating
Mousse	1	1	0.1
Pie	3	2	0.5
Ice Cream	2	6	0.2
Sorbet	3	4	0.4
Cheesecake	5	3	0.9



$W^{[l]}, b^{[l]}$: weights and bias on layer l
 $z^{[l]}$: weighted sum of inputs on layer l ,
 $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$
 $g^{[l]}$: activation function on layer l
 $a^{[l]}$: output of activation function on layer l ,
 $a^{[l]} = g(z^{[l]})$

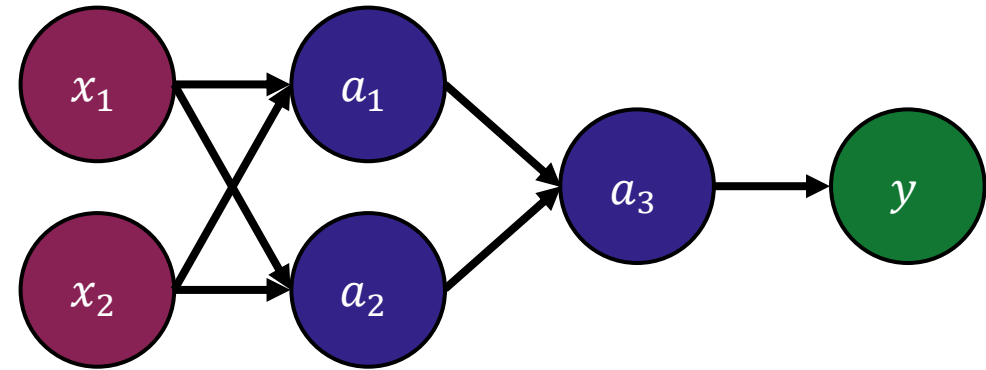
Backpropagation

Based on our result from the **Ice Cream** data point, update the NN weights using **stochastic gradient descent**.

We'll compute the partial derivative of the loss function: $L_2(y, \hat{y}) = (y - \hat{y})^2$

$$\frac{\partial L_2(\mathbf{w})}{\partial w_i}$$

Food	x_1	x_2	Rating
Mousse	1	1	0.1
Pie	3	2	0.5
Ice Cream	2	6	0.2
Sorbet	3	4	0.4
Cheesecake	5	3	0.9



$W^{[l]}, b^{[l]}$: weights and bias on layer l
 $z^{[l]}$: weighted sum of inputs on layer l ,
 $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$
 $g^{[l]}$: activation function on layer l
 $a^{[l]}$: output of activation function on layer l ,
 $a^{[l]} = g(z^{[l]})$

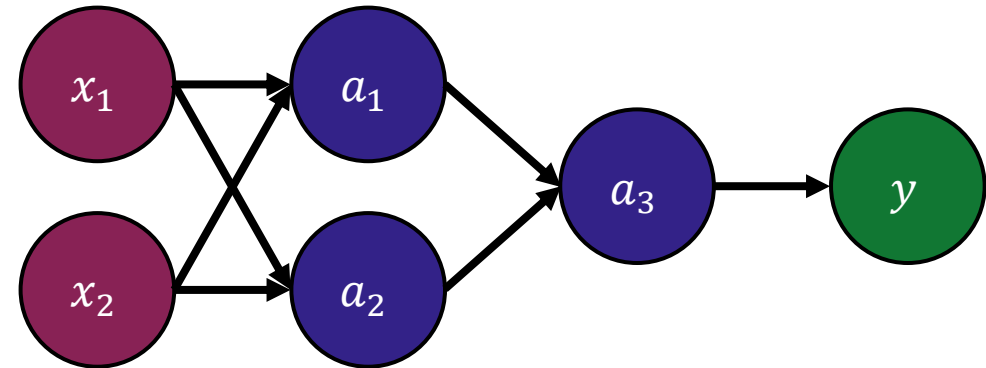
Backpropagation

Based on our result from the **Ice Cream** data point, update the NN weights using **stochastic gradient descent**.

We'll compute the partial derivative of the loss function: $L_2(y, \hat{y}) = (y - \hat{y})^2$

$$\frac{\partial L_2(\mathbf{w})}{\partial w_i} = \underbrace{\frac{\partial L_2(\mathbf{w})}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w_i}}_{\text{Chain rule}}$$

Food	x_1	x_2	Rating
Mousse	1	1	0.1
Pie	3	2	0.5
Ice Cream	2	6	0.2
Sorbet	3	4	0.4
Cheesecake	5	3	0.9



$W^{[l]}, b^{[l]}$: weights and bias on layer l
 $z^{[l]}$: weighted sum of inputs on layer l ,
 $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$
 $g^{[l]}$: activation function on layer l
 $a^{[l]}$: output of activation function on layer l ,
 $a^{[l]} = g(z^{[l]})$

Backpropagation

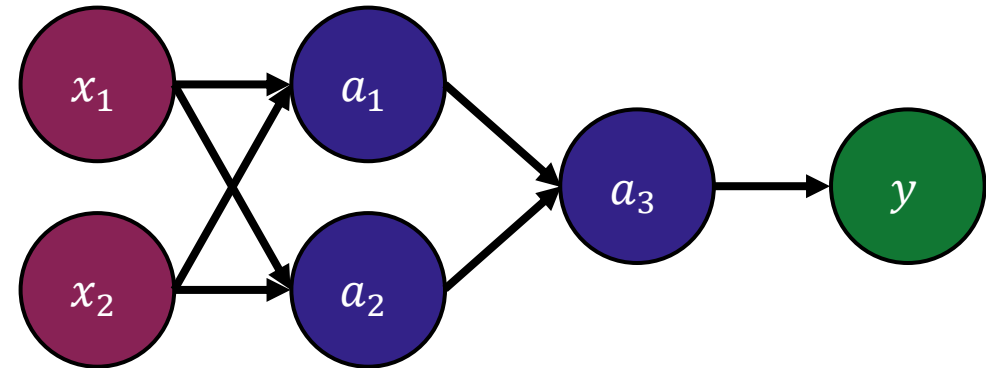
Based on our result from the **Ice Cream** data point, update the NN weights using **stochastic gradient descent**.

We'll compute the partial derivative of the loss function: $L_2(y, \hat{y}) = (y - \hat{y})^2$

$$\frac{\partial L_2(\mathbf{w})}{\partial w_i} = \frac{\partial L_2(\mathbf{w})}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w_i}$$

Chain rule
Explain: why this chain?

Food	x_1	x_2	Rating
Mousse	1	1	0.1
Pie	3	2	0.5
Ice Cream	2	6	0.2
Sorbet	3	4	0.4
Cheesecake	5	3	0.9



$W^{[l]}, b^{[l]}$: weights and bias on layer l
 $z^{[l]}$: weighted sum of inputs on layer l ,
 $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$
 $g^{[l]}$: activation function on layer l
 $a^{[l]}$: output of activation function on layer l ,
 $a^{[l]} = g(z^{[l]})$

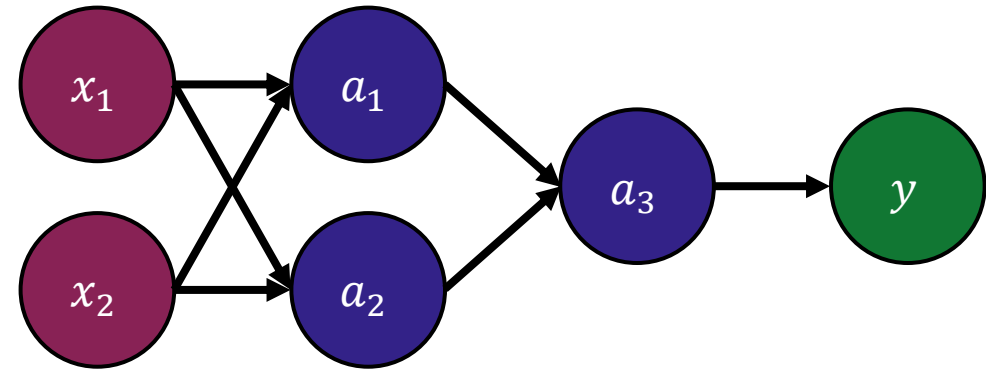
Backpropagation

Based on our result from the **Ice Cream** data point, update the NN weights using **stochastic gradient descent**.

We'll compute the partial derivative of the loss function: $L_2(y, \hat{y}) = (y - \hat{y})^2$

$$\frac{\partial L_2(\mathbf{w})}{\partial w_i} = \frac{\partial L_2(\mathbf{w})}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w_i}$$

Food	x_1	x_2	Rating
Mousse	1	1	0.1
Pie	3	2	0.5
Ice Cream	2	6	0.2
Sorbet	3	4	0.4
Cheesecake	5	3	0.9



$W^{[l]}, b^{[l]}$: weights and bias on layer l
 $z^{[l]}$: weighted sum of inputs on layer l ,
 $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$
 $g^{[l]}$: activation function on layer l
 $a^{[l]}$: output of activation function on layer l ,
 $a^{[l]} = g(z^{[l]})$

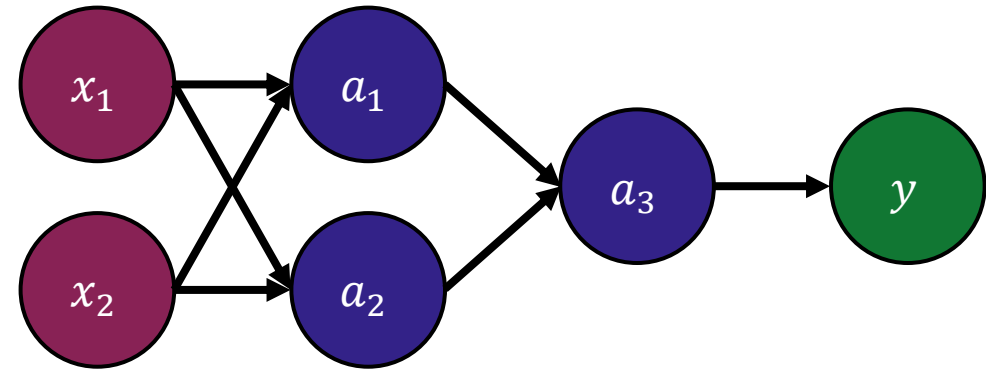
Backpropagation

Based on our result from the **Ice Cream** data point, update the NN weights using **stochastic gradient descent**.

We'll compute the partial derivative of the loss function: $L_2(y, \hat{y}) = (y - \hat{y})^2$

$$\frac{\partial L_2(\mathbf{w})}{\partial w_i} = \frac{\partial}{\partial \hat{y}} (y - \hat{y})^2 \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w_i}$$

Food	x_1	x_2	Rating
Mousse	1	1	0.1
Pie	3	2	0.5
Ice Cream	2	6	0.2
Sorbet	3	4	0.4
Cheesecake	5	3	0.9



$W^{[l]}, b^{[l]}$: weights and bias on layer l
 $z^{[l]}$: weighted sum of inputs on layer l ,
 $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$
 $g^{[l]}$: activation function on layer l
 $a^{[l]}$: output of activation function on layer l ,
 $a^{[l]} = g(z^{[l]})$

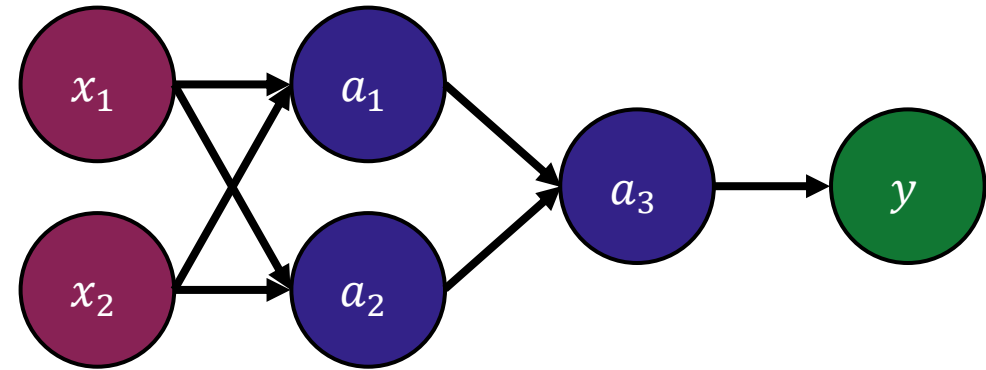
Backpropagation

Based on our result from the **Ice Cream** data point, update the NN weights using **stochastic gradient descent**.

We'll compute the partial derivative of the loss function: $L_2(y, \hat{y}) = (y - \hat{y})^2$

$$\frac{\partial L_2(\mathbf{w})}{\partial w_i} = -2(y - \hat{y}) \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w_i}$$

Food	x_1	x_2	Rating
Mousse	1	1	0.1
Pie	3	2	0.5
Ice Cream	2	6	0.2
Sorbet	3	4	0.4
Cheesecake	5	3	0.9



$W^{[l]}, b^{[l]}$: weights and bias on layer l
 $z^{[l]}$: weighted sum of inputs on layer l ,
 $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$
 $g^{[l]}$: activation function on layer l
 $a^{[l]}$: output of activation function on layer l ,
 $a^{[l]} = g(z^{[l]})$

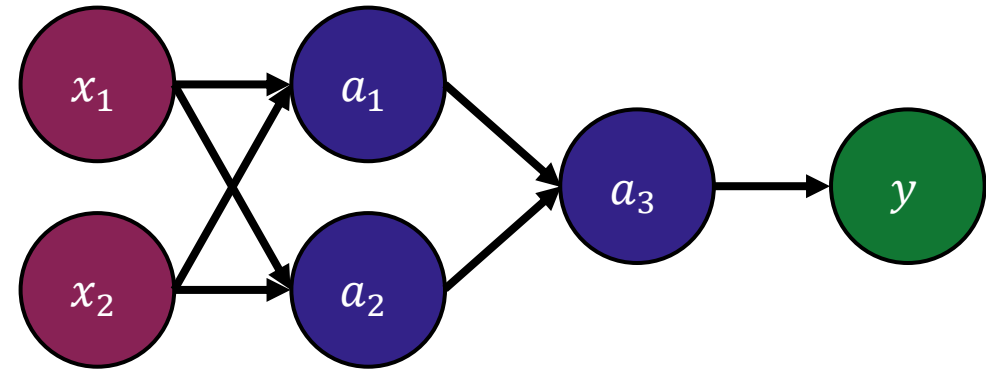
Backpropagation

Based on our result from the **Ice Cream** data point, update the NN weights using **stochastic gradient descent**.

We'll compute the partial derivative of the loss function: $L_2(y, \hat{y}) = (y - \hat{y})^2$

$$\frac{\partial L_2(\mathbf{w})}{\partial w_i} = -2(y - \hat{y}) \cdot \frac{\partial}{\partial z} \frac{1}{1 + e^{-z}} \cdot \frac{\partial z}{\partial w_i}$$

Food	x_1	x_2	Rating
Mousse	1	1	0.1
Pie	3	2	0.5
Ice Cream	2	6	0.2
Sorbet	3	4	0.4
Cheesecake	5	3	0.9



$W^{[l]}, b^{[l]}$: weights and bias on layer l
 $z^{[l]}$: weighted sum of inputs on layer l ,
 $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$
 $g^{[l]}$: activation function on layer l
 $a^{[l]}$: output of activation function on layer l ,
 $a^{[l]} = g(z^{[l]})$

Backpropagation

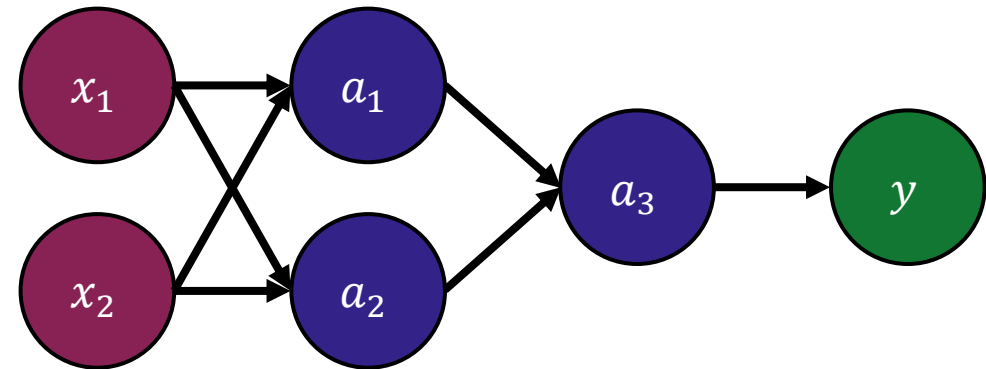
Based on our result from the **Ice Cream** data point, update the NN weights using **stochastic gradient descent**.

We'll compute the partial derivative of the loss function: $L_2(y, \hat{y}) = (y - \hat{y})^2$

$$\frac{\partial L_2(\mathbf{w})}{\partial w_i} = -2(y - \hat{y})\hat{y}(1 - \hat{y}) \cdot \frac{\partial z}{\partial w_i}$$

We'll give you the derivative of the activation function. Some of these can be tricky, but it can be a fun challenge to try this derivation on your own!

Food	x_1	x_2	Rating
Mousse	1	1	0.1
Pie	3	2	0.5
Ice Cream	2	6	0.2
Sorbet	3	4	0.4
Cheesecake	5	3	0.9



$W^{[l]}, b^{[l]}$: weights and bias on layer l
 $z^{[l]}$: weighted sum of inputs on layer l ,
 $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$
 $g^{[l]}$: activation function on layer l
 $a^{[l]}$: output of activation function on layer l ,
 $a^{[l]} = g(z^{[l]})$

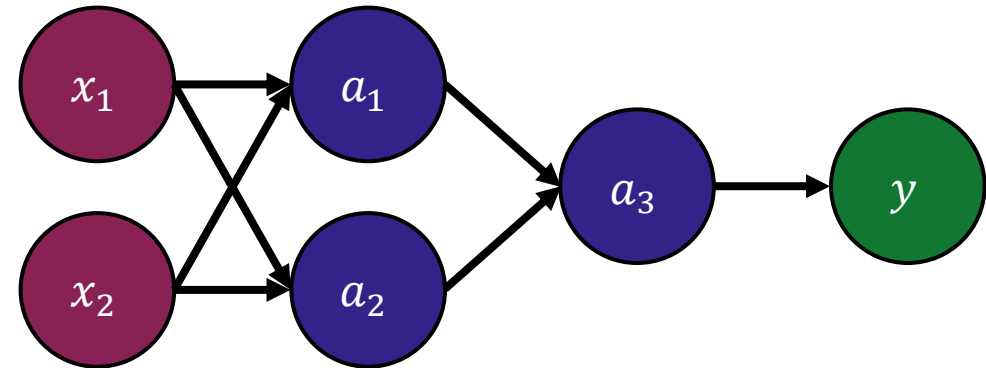
Backpropagation

Based on our result from the **Ice Cream** data point, update the NN weights using **stochastic gradient descent**.

We'll compute the partial derivative of the loss function: $L_2(y, \hat{y}) = (y - \hat{y})^2$

$$\frac{\partial L_2(\mathbf{w})}{\partial w_i} = -2(y - \hat{y})\hat{y}(1 - \hat{y}) \cdot \frac{\partial z}{\partial w_i}$$

Food	x_1	x_2	Rating
Mousse	1	1	0.1
Pie	3	2	0.5
Ice Cream	2	6	0.2
Sorbet	3	4	0.4
Cheesecake	5	3	0.9



$W^{[l]}, b^{[l]}$: weights and bias on layer l
 $z^{[l]}$: weighted sum of inputs on layer l ,
 $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$
 $g^{[l]}$: activation function on layer l
 $a^{[l]}$: output of activation function on layer l ,
 $a^{[l]} = g(z^{[l]})$

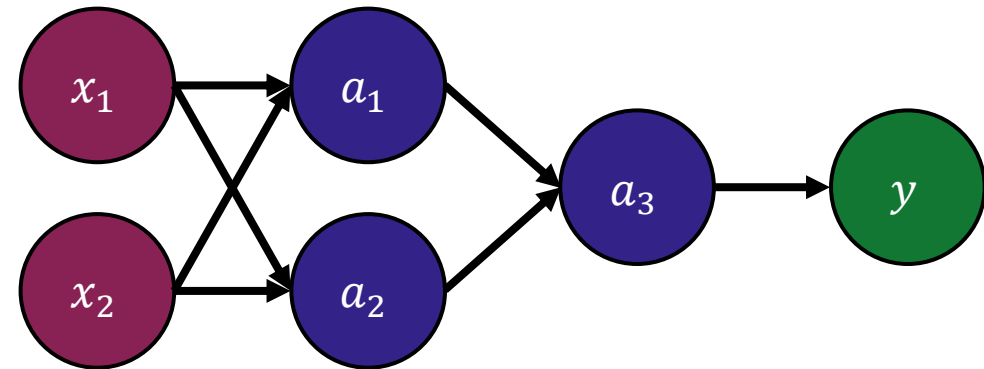
Backpropagation

Based on our result from the **Ice Cream** data point, update the NN weights using **stochastic gradient descent**.

We'll compute the partial derivative of the loss function: $L_2(y, \hat{y}) = (y - \hat{y})^2$

$$\frac{\partial L_2(\mathbf{w})}{\partial w_i} = -2(y - \hat{y})\hat{y}(1 - \hat{y}) \cdot \frac{\partial}{\partial w_i} \left(\sum w_i x_i \right)$$

Food	x_1	x_2	Rating
Mousse	1	1	0.1
Pie	3	2	0.5
Ice Cream	2	6	0.2
Sorbet	3	4	0.4
Cheesecake	5	3	0.9



$W^{[l]}, b^{[l]}$: weights and bias on layer l
 $z^{[l]}$: weighted sum of inputs on layer l ,
 $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$
 $g^{[l]}$: activation function on layer l
 $a^{[l]}$: output of activation function on layer l ,
 $a^{[l]} = g(z^{[l]})$

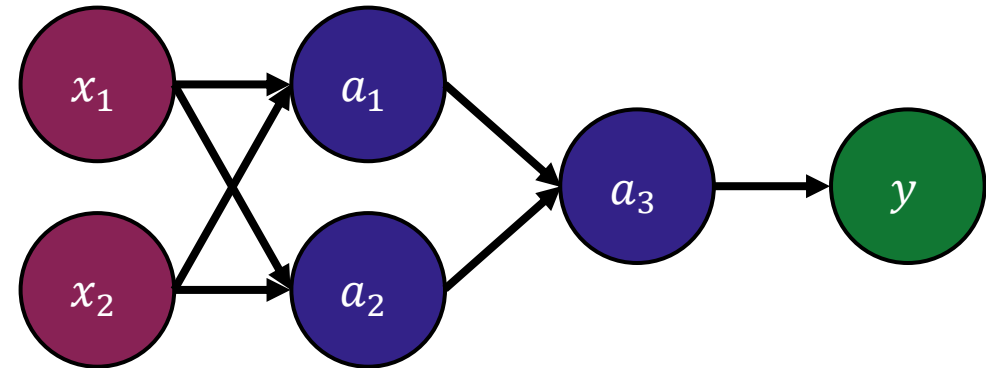
Backpropagation

Based on our result from the **Ice Cream** data point, update the NN weights using **stochastic gradient descent**.

We'll compute the partial derivative of the loss function: $L_2(y, \hat{y}) = (y - \hat{y})^2$

$$\frac{\partial L_2(\mathbf{w})}{\partial w_i} = -2(y - \hat{y})\hat{y}(1 - \hat{y})x_i$$

Food	x_1	x_2	Rating
Mousse	1	1	0.1
Pie	3	2	0.5
Ice Cream	2	6	0.2
Sorbet	3	4	0.4
Cheesecake	5	3	0.9



$W^{[l]}, b^{[l]}$: weights and bias on layer l
 $z^{[l]}$: weighted sum of inputs on layer l ,
 $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$
 $g^{[l]}$: activation function on layer l
 $a^{[l]}$: output of activation function on layer l ,
 $a^{[l]} = g(z^{[l]})$

Backpropagation

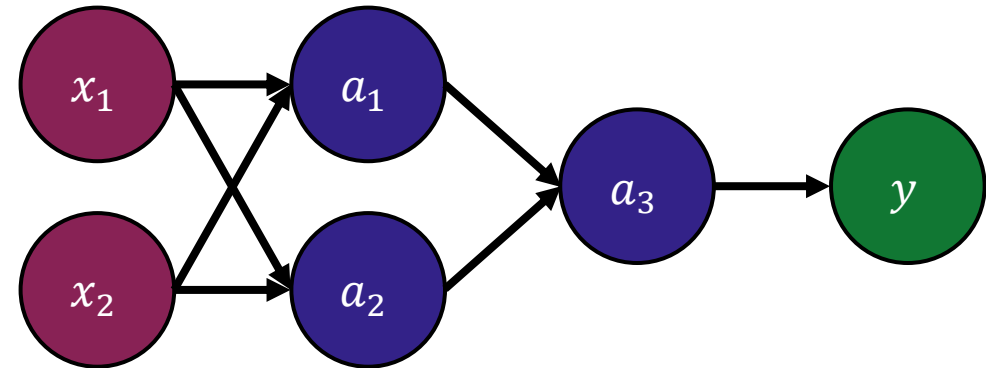
Based on our result from the **Ice Cream** data point, update the NN weights using **stochastic gradient descent**.

Plugging back into our weight update rule:

$$w_i \leftarrow w_i - \alpha \frac{\partial}{\partial w_i} L_2(\mathbf{w})$$

$$\frac{\partial L_2(\mathbf{w})}{\partial w_i} = -2(y - \hat{y})\hat{y}(1 - \hat{y})x_i$$

Food	x_1	x_2	Rating
Mousse	1	1	0.1
Pie	3	2	0.5
Ice Cream	2	6	0.2
Sorbet	3	4	0.4
Cheesecake	5	3	0.9



$W^{[l]}, b^{[l]}$: weights and bias on layer l
 $z^{[l]}$: weighted sum of inputs on layer l ,
 $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$
 $g^{[l]}$: activation function on layer l
 $a^{[l]}$: output of activation function on layer l ,
 $a^{[l]} = g(z^{[l]})$

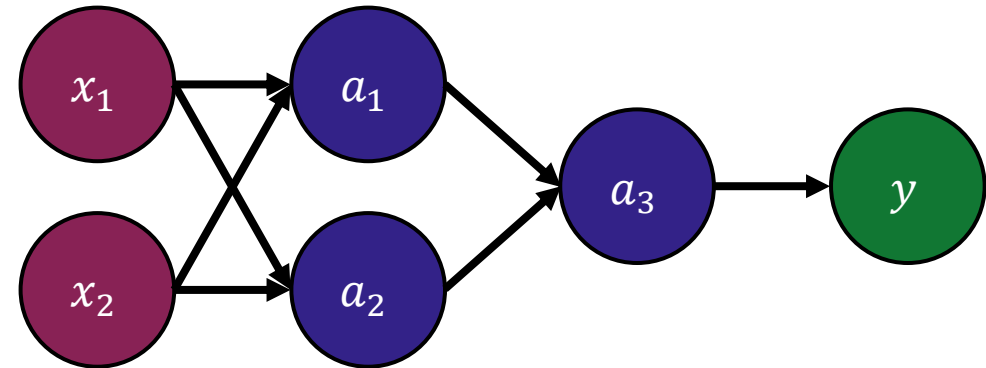
Backpropagation

Based on our result from the **Ice Cream** data point, update the NN weights using **stochastic gradient descent**.

Plugging back into our weight update rule:

$$w_i \leftarrow w_i + 2\alpha (y - \hat{y})\hat{y}(1 - \hat{y})x_i$$

Food	x_1	x_2	Rating
Mousse	1	1	0.1
Pie	3	2	0.5
Ice Cream	2	6	0.2
Sorbet	3	4	0.4
Cheesecake	5	3	0.9



$W^{[l]}, b^{[l]}$: weights and bias on layer l
 $z^{[l]}$: weighted sum of inputs on layer l ,
 $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$
 $g^{[l]}$: activation function on layer l
 $a^{[l]}$: output of activation function on layer l ,
 $a^{[l]} = g(z^{[l]})$

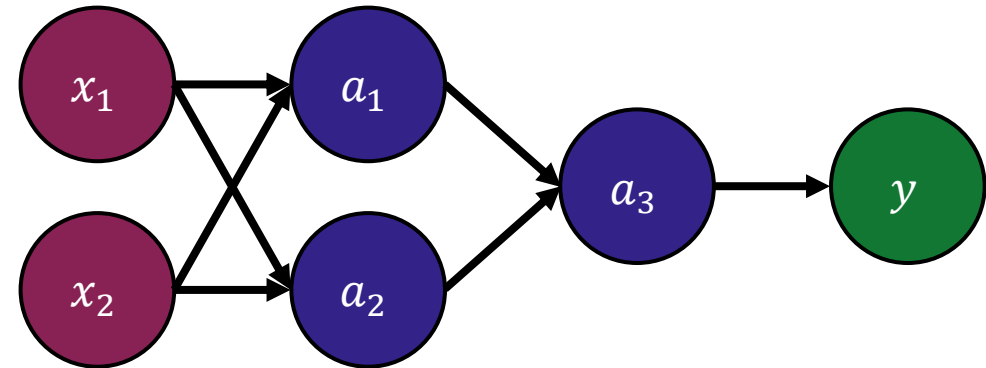
Backpropagation

Based on our result from the **Ice Cream** data point, update the NN weights using **stochastic gradient descent**.

We can start computing our new weights!

$$w_i \leftarrow w_i + 2\alpha (y - \hat{y})\hat{y}(1 - \hat{y})x_i$$

Food	x_1	x_2	Rating
Mousse	1	1	0.1
Pie	3	2	0.5
Ice Cream	2	6	0.2
Sorbet	3	4	0.4
Cheesecake	5	3	0.9



$W^{[l]}, b^{[l]}$: weights and bias on layer l

$z^{[l]}$: weighted sum of inputs on layer l ,

$$z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$$

$g^{[l]}$: activation function on layer l

$a^{[l]}$: output of activation function on layer l ,

$$a^{[l]} = g(z^{[l]})$$

Backpropagation

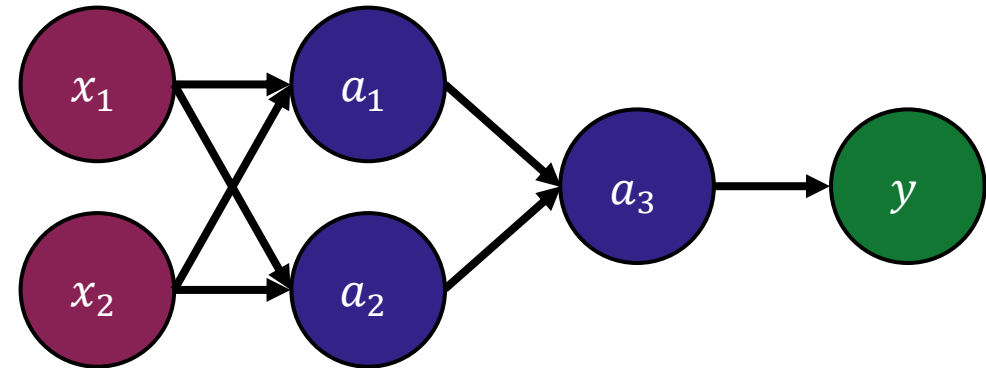
Based on our result from the **Ice Cream** data point, update the NN weights using **stochastic gradient descent**.

How does this change if w_i is in a hidden layer?

$$w_i \leftarrow w_i + 2\alpha (y - \hat{y})\hat{y}(1 - \hat{y})x_i$$

When we are one layer removed from the output, we must account for the contribution of the node—say hidden node a_1 —on the output.

Food	x_1	x_2	Rating
Mousse	1	1	0.1
Pie	3	2	0.5
Ice Cream	2	6	0.2
Sorbet	3	4	0.4
Cheesecake	5	3	0.9



$W^{[l]}, b^{[l]}$: weights and bias on layer l
 $z^{[l]}$: weighted sum of inputs on layer l ,
 $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$
 $g^{[l]}$: activation function on layer l
 $a^{[l]}$: output of activation function on layer l ,
 $a^{[l]} = g(z^{[l]})$

Backpropagation

Based on our result from the **Ice Cream** data point, update the NN weights using **stochastic gradient descent**.

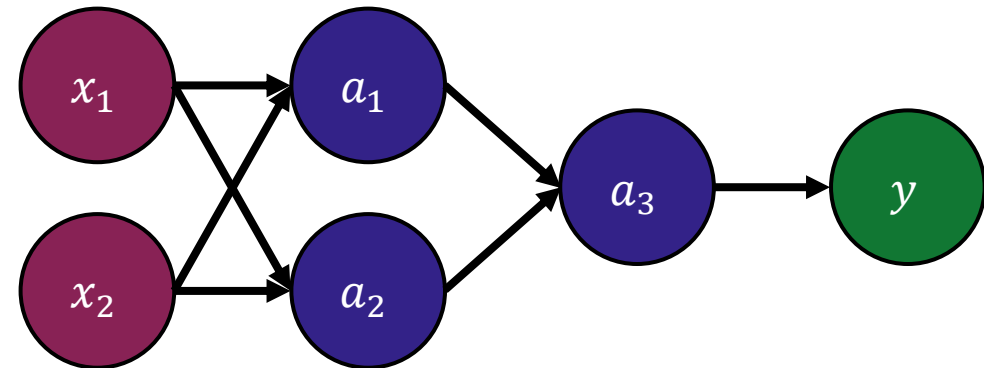
How does this change if w_i is in a hidden layer?

$$w_i \leftarrow w_i + 2\alpha (y - \hat{y})\hat{y}(1 - \hat{y})x_i$$

When we are one layer removed from the output, we must account for the contribution of the node—say hidden node a_1 —on the output.

In particular, we'd also multiply by $w_1^{[2]} a_1$ to show how x_i is moved through layer 1 for prediction $\hat{y}$

Food	x_1	x_2	Rating
Mousse	1	1	0.1
Pie	3	2	0.5
Ice Cream	2	6	0.2
Sorbet	3	4	0.4
Cheesecake	5	3	0.9



$W^{[l]}, b^{[l]}$: weights and bias on layer l
 $z^{[l]}$: weighted sum of inputs on layer l ,
 $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$
 $g^{[l]}$: activation function on layer l
 $a^{[l]}$: output of activation function on layer l ,
 $a^{[l]} = g(z^{[l]})$

Questions?

References

- Russell and Norvig, 4th Edition
 - Chapter 19.6 (gradient descent)
 - Chapter 21.1 (neural networks)
- [Vectorization in Fully-Connected Neural Networks](#)
- Additional sources for backpropagation notation:
 - [Understanding Vectorized Implementation of Neural Networks](#)
 - [Vectorization Operations](#)